Bot helpers: a beginner's reference

Your bot is one Python file in the `bots` folder. It inherits simple chess helpers from `BaseBot`, so you can start without learning the chess library.

Quick start

Save this as `bots/my_bot.py`:

```
from engine.base_bot import BaseBot

class Bot(BaseBot):
    name = "My First Bot"

    def choose_move(self, board):
        moves = self.get_legal_moves(board)
        for move in moves:
            if self.is_check_mate(board, move):
                return move
        return moves[0]
```

Line by line:

1. Import the bot base class supplied by the app.
2. Leave a blank line to make the code easier to read.
3. Start the class named `Bot`; the loader looks for this exact name.
4. Give your bot a name for the menus and tournament table.
5. Leave a blank line inside the class.
6. Define the method the game calls to ask for a move.
7. Ask for every legal move in the position.
8. Look through the possible moves one at a time.
9. Check whether the move ends the game with checkmate.
10. Return a winning move immediately when you find one.
11. If no checkmate was found, return the first legal move.

How moves are written

A move is written with the starting square followed by the destination square. Files are letters `a` to `h` from left to right. Ranks are numbers `1` to `8` from White's side to Black's side.

```
    a   b   c   d   e   f   g   h
  +---+---+---+---+---+---+---+---+
8 |   |   |   |   |   |   |   |   | 8
7 |   |   |   |   |   |   |   |   | 7
6 |   |   |   |   |   |   |   |   | 6
5 |   |   |   |   |   |   |   |   | 5
4 |   |   |   |   |   |   |   |   | 4
3 |   |   |   |   |   |   |   |   | 3
2 |   |   |   |   |   |   |   |   | 2
1 |   |   |   |   |   |   |   |   | 1
  +---+---+---+---+---+---+---+---+
    a   b   c   d   e   f   g   h
```

For example, `e2e4` moves a piece from `e2` to `e4`. A promotion adds the piece letter at the end: `e7e8q` promotes to a queen (`q`, rook `r`, bishop `b`, or knight `n`). Castling is written as the king's move, such as `e1g1` for White's short castle.

Use lower-case letters with no dash. Write `e2e4`, not `E2E4` or `e2-e4`.

Helper summary

Helper	What it does	What you give it	What it gives back
`is_legal`	Checks whether a move can be played now.	`board`, move string or move object	`True` or `False`
`is_check`	Checks whether a move attacks the other king.	`board`, move string or move object	`True` or `False`
`is_check_mate`	Checks whether a move wins by checkmate.	`board`, move string or move object	`True` or `False`
`get_legal_moves`	Lists every move you may play now.	`board`	A list of move strings
`is_capture`	Checks whether a move takes an opponent's piece.	`board`, move string or move object	`True` or `False`
`piece_value`	Gives a piece a simple value.	`board`, square such as `"d4"`	A number from 0 to 9

These helpers never change the board you give them. The move-checking helpers return `False` for an illegal move or a badly written move string.

`is_legal(board, move)`

Checks whether the move is allowed in this position.

- Give it the current `board` and a move such as `"e2e4"`.
- It returns `True` or `False`.

```
if self.is_legal(board, "e2e4"):
    print("That move is allowed")
```

It prints `That move is allowed` at the start of a game.

Common mistake: Call it as `self.is_legal(...)`. The move must use lower case with no dash, for example `"e2e4"`.

`is_check(board, move)`

Checks whether the move puts the other player's king in check.

- Give it the current `board` and a candidate move.
- It returns `True` or `False`.

```
print(self.is_check(board, "e2e4"))
```

At the start of a game this prints `False`.

Common mistake: This checks the position *after* your move. A move that does not give check returns `False`, even when the other king is already under attack in some different position.

`is_check_mate(board, move)`

Checks whether the move wins the game by checkmate.

- Give it the current `board` and a candidate move.
- It returns `True` or `False`.

```
for move in self.get_legal_moves(board):
    if self.is_check_mate(board, move):
        print("I found a winning move:", move)
```

It prints a message only when a winning move exists.

Common mistake: Check every legal move; do not assume a move is mate just because it gives check.

`get_legal_moves(board)`

Returns all moves that are allowed in the current position.

- Give it the current `board`.
- It returns a list of strings, such as `["g1h3", "g1f3", ...]`.

```
moves = self.get_legal_moves(board)
print("I have", len(moves), "possible moves")
```

At the start of a game it prints `I have 20 possible moves`.

Common mistake: Use one item from this list as your returned move. Do not try to change the list into a board square.

`is_capture(board, move)`

Checks whether a move captures an opponent's piece.

- Give it the current `board` and a candidate move.
- It returns `True` or `False`. It also recognises en passant captures.

```
for move in self.get_legal_moves(board):
    if self.is_capture(board, move):
        print("This move captures:", move)
```

It prints one line for every legal capture.

Common mistake: A normal move to an empty square is not a capture.

`piece_value(board, square)`

Returns a simple score for the piece on one square. Pawns are worth `1`, knights and bishops `3`, rooks `5`, queens `9`, and kings `0`. Empty or invalid squares are worth `0`.

- Give it the current `board` and a square name such as `"d4"`.
- It returns a whole number.

```
print(self.piece_value(board, "e2"))
```

At the start of a game this prints `1`, because a pawn is on `e2`.

Common mistake: Give it one square (`"e2"`), not a move (`"e2e4"`).

What `choose_move` must do

The game calls `choose_move(board)` when it is your turn. The board is a copy: you can inspect it, but changing it will not change the real game. Return either a legal move string such as `"e2e4"` or a move object. Returning one of the strings from `get_legal_moves(board)` is a good first approach.

If your bot returns an illegal move, returns something other than a move, or crashes, it loses that game. The app shows the reason and saves it in the game record. A safety limit stops a bot that does not respond; it is not a chess clock and is not passed to your method.

Next steps

1. Use `get_legal_moves` and `is_capture` to prefer taking pieces.
2. Use `piece_value` to choose the most valuable piece you can capture.
3. Use `is_check` and `is_check_mate` to look for threats and winning moves.
4. Try looking one or two moves ahead and compare the material values.